Computer Architecture Laboratory

Milestone 1: Warm-up

Status: Version 1.0
Author: Khai Pham
Vy Luong
Date: 2026-Sep-25

1. Overview

Milestone 1 introduces several fundamental RTL components that will be reused in later milestones. The exercises are intentionally small so that students can become familiar with RTL design, simulation, and hardware specifications before integrating these components into a processor.

The milestone also establishes common interfaces and behaviors for several data-path and memory blocks to simplify integration in later milestones.

2. Objective

- Become familiar with the RTL design and simulation workflow.
- Practice basic combinational and sequential logic design.
- Understand and follow SystemVerilog coding guidelines.
- Implement basic datapath and storage components.
- Build reusable RTL blocks for later processor milestones.

3. SystemVerilog Coding Style Guidelines

This section introduces the SystemVerilog coding practices expected throughout the course. The purpose of these guidelines is to keep RTL code consistent, readable, and maintainable.

Students will frequently revisit their code for debugging, verification, modification, and integration. Following a consistent coding style makes these tasks easier and reduces avoidable mistakes.

The guidelines generally do not change the intended functionality of a design. Instead, they promote clear RTL structure and good design practice.

Students are expected to follow these rules consistently throughout the course.

In addition, students are encouraged to follow the lowRISC Verilog/SystemVerilog Coding Style Guide where applicable.

4. Problem

In these exercises, students implement several basic operations directly in RTL to understand their hardware structure before using equivalent HDL operators such as `+`, `-`, `<<`, and `>>`.

Register files and memories are also common storage structures. Students therefore implement simple HDL memory models that can be reused to verify and integrate later processor milestones before the final memory implementation is introduced.

4.1. Exercise 1: Adder

The adder performs 32-bit addition and subtraction. It takes two 32-bit operands and two control inputs, and produces a 32-bit result together with a carry-out bit.

The control signal `i_sub` selects between addition and subtraction. The signal `i_unsigned` indicates whether the operands are interpreted as unsigned values or as two's complement signed values.

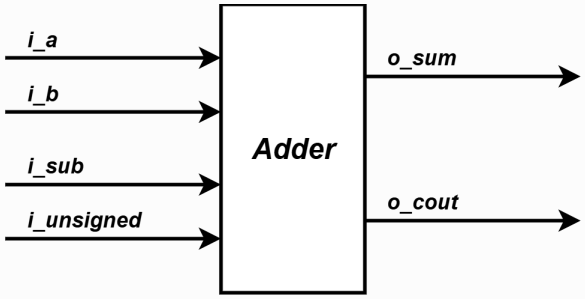


Table 1: Input and output signals of the adder.

Signal name	Width	Direction	Description
`i_a`	32	input	First operand.
`i_b`	32	input	Second operand.
`i_sub`	1	input	Selects subtraction when high and addition when low.
`i_unsigned`	1	input	Selects unsigned interpretation when high.
`o_sum`	32	output	32-bit result of the selected operation.
`o_cout`	1	output	Carry-out status bit.

? PROBLEM

Students research a adder architecture and implement it in RTL according to the specified interface. The `+` and `-` operators are prohibited. Students are also encouraged to write their own testbench to verify the design.

4.2. Exercise 2: Magnitude Comparator

The magnitude comparator compares two 32-bit operands and reports their ordering through three output signals. The control signal `i_unsigned` selects whether the operands are interpreted as unsigned values or as two's complement signed values.

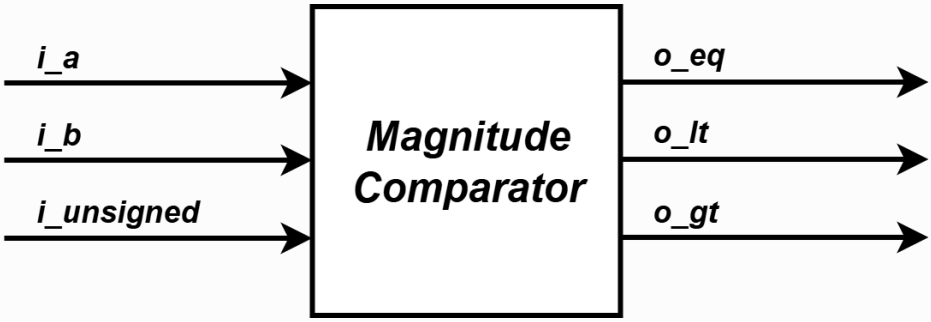


Figure 2: Comparator input/output signal specification

Table 2: Input and output signals of the magnitude comparator.

Signal name	Width	Direction	Description
`i_a`	32	input	First operand of the comparison.
`i_b`	32	input	Second operand of the comparison.
`i_unsigned`	1	input	Selects unsigned comparison when high and signed comparison when low.
`o_eq`	1	output	High when `i_a = i_b`.
`o_lt`	1	output	High when `i_a < i_b`.
`o_gt`	1	output	High when `i_a > i_b`.

Exactly one of `o_eq`, `o_lt`, and `o_gt` shall be high for any valid pair of input operands.

> **? PROBLEM**
>
> Students research a suitable magnitude comparator architecture and implement it in RTL according to the specified interface. The comparison operators `<`, `>`, `<=`, and `>=` are prohibited. Students are also encouraged to write their own testbench to verify the design.

4.3. Exercise 3: Shifter

The shifter is a combinational block that shifts a 32-bit operand by a variable number of bit positions. The shift amount is specified by `i_shamt`, while `i_mode` selects the shift direction and fill behavior.

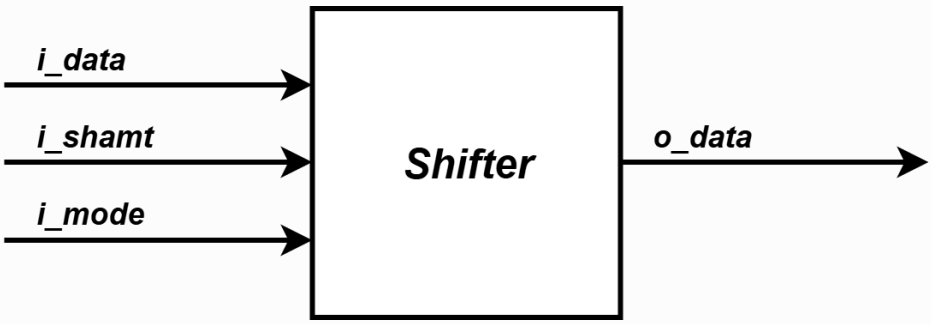


Figure 3: Shifter input/output signal specification

Table 3: Input and output signals of the shifter.

Signal name	Width	Direction	Description
`i_data`	32	input	Operand to be shifted.
`i_shamt`	5	input	Shift amount from 0 to 31 bit positions.
`i_mode`	2	input	Selects the shift operation.
`o_data`	32	output	Shifted result.

The shifter supports logical left, logical right, and arithmetic right shifts.

Table 4: Operating modes of the shifter selected by `i_mode`.

`i_mode`	Operation	Description
`2'b00`	Logical right shift	Shifts toward the least significant bit and fills the upper bits with zero.
`2'b01`	Logical left shift	Shifts toward the most significant bit and fills the lower bits with zero.
`2'b10`	Arithmetic right shift	Shifts toward the least significant bit and fills the upper bits with the original sign bit.
`2'b11`	Reserved	`o_data` shall be set to a defined safe value.

? PROBLEM

Students research a suitable shifter architecture and implement it in RTL according to the specified interface. The shift operators `<<`, `>>`, and `>>>` are prohibited. Students are also encouraged to write their own testbench to verify the design.

4.4. Exercise 4: Arithmetic Logic Unit (ALU)

The ALU combines the arithmetic, comparison, and shift functions from the previous exercises into a single combinational datapath. It takes two 32-bit operands and a 4-bit operation code, and produces one 32-bit result.

Table 5: Input and output signals of the ALU.

Signal name	Width	Direction	Description
`i_op_a`	32	input	First operand of the ALU operation.
`i_op_b`	32	input	Second operand of the ALU operation.
`i_alu_op`	4	input	Selects the ALU operation.
`o_alu_data`	32	output	Result of the selected operation.

The 4-bit control signal **`i_alu_op`** selects one of the ten supported operations listed in Table 6. Unused encodings shall drive **`o_alu_data`** to a defined safe value.

Table 6: ALU operations, where **`a`** denotes **`i_op_a`** and **`b`** denotes **`i_op_b`**.

alu_op	Operation	Description
ADD	`a + b`	Addition.
SUB	`a - b`	Subtraction.
SLT	`(a < b) ? 1 : 0`	Signed less-than comparison.
SLTU	`(a < b) ? 1 : 0`	Unsigned less-than comparison.
XOR	`a ^ b`	Bitwise exclusive OR.
OR	`a | b`	Bitwise OR.
AND	`a & b`	Bitwise AND.
SLL	`a << b[4:0]`	Logical left shift.
SRL	`a >> b[4:0]`	Logical right shift.
SRA	`a >>> b[4:0]`	Arithmetic right shift.

For **`SLT`** and **`SLTU`**, the comparison result is placed in bit 0 of **`o_alu_data`**, while bits 31:1 are set to zero.

The adder, comparator, and shifter from the previous exercises shall be reused inside the ALU. Each block is instantiated once and shared by the operations that require it. Logic operations may be implemented directly in the ALU.

? PROBLEM

Students define the encoding of **`i_alu_op`**, integrate the previously implemented adder, comparator, and shifter, and implement the logic and result-selection circuitry required for the ALU. Students are also encouraged to write their own testbench to verify the design.

4.5. Exercise 5: Register File

The register file contains 32 registers, **x0** to **x31**, and provides two asynchronous read ports and one synchronous write port. The **i_rd_** prefix refers to the destination register of an instruction.

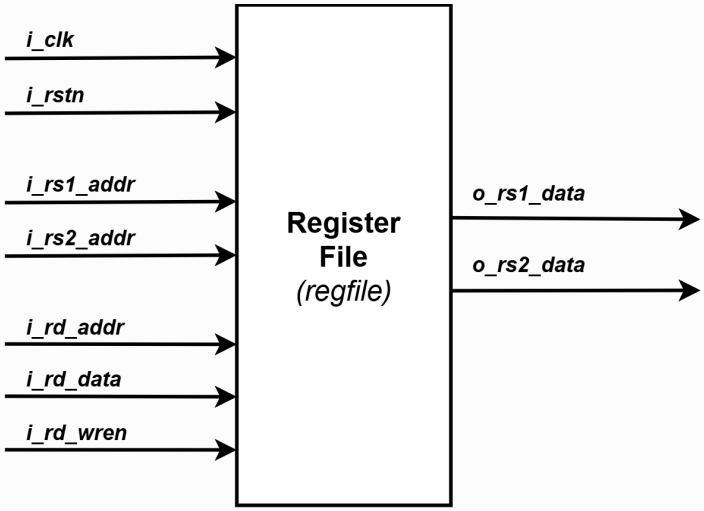


Figure 4: Register file input/output signal specification

Table 7: Input and output signals of the register file.

Signal name	Width	Direction	Description
i_clk	1	input	Clock of the write port.
i_rstn	1	input	Asynchronous active-low reset.
i_rs1_addr	5	input	Address of the first read port.
i_rs2_addr	5	input	Address of the second read port.
i_rd_addr	5	input	Address of the write port.
i_rd_data	32	input	Data to be written.
i_rd_wren	1	input	Write enable.
o_rs1_data	32	output	Data read from the register selected by i_rs1_addr.
o_rs2_data	32	output	Data read from the register selected by i_rs2_addr.

The two read ports are asynchronous. **o_rs1_data** and **o_rs2_data** follow their respective address inputs through combinational logic and do not require a clock edge.

The write port is synchronous. On the rising edge of **i_clk**, when **i_rd_wren** is high, **i_rd_data** is written to the register selected by **i_rd_addr**.

---

*Register **x0** is fixed to zero. Writes to address zero shall be ignored, and both read ports shall return zero whenever address zero is selected.*

*When **i_rstn** is asserted low, all registers shall be cleared to zero asynchronously.*

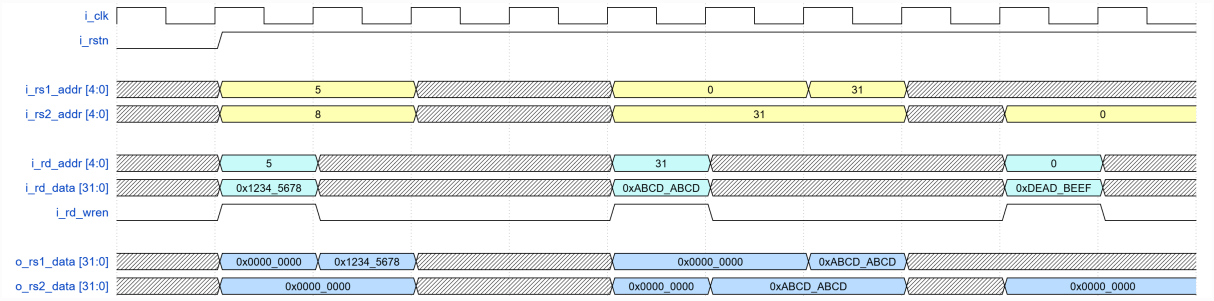


Figure 5: Example register file behavior.

? PROBLEM

Students implement the register file in RTL according to the specified read, write, reset, and `x0` behavior. Students are also encouraged to write their own testbench to verify the design.

4.6. Exercise 6: Data Memory with Asynchronous Read

In this exercise, students implement a byte-addressed memory model with a synchronous write port and an asynchronous read port. The model will be reused in later processor exercises.

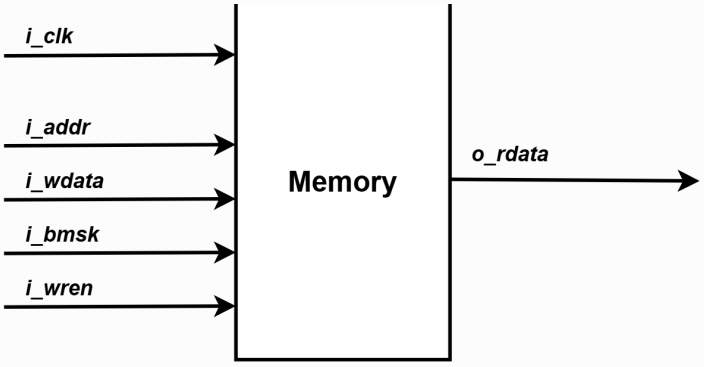


Figure 6: Asynchronous read memory input/output signal specification

Table 8: Input and output signals of the asynchronous read memory.

Signal name	Width	Direction	Description
`i_clk`	1	input	Clock of the write port.
`i_addr`	32	input	Byte address of the memory access.
`i_wdata`	32	input	Data to be written.
`i_bmsk`	4	input	Byte enable mask for the four byte lanes.
`i_wren`	1	input	Write enable.
`o_rdata`	32	output	Data stored at the addressed word.

The memory stores 32-bit words using byte addresses. Each word contains four byte lanes, with **`i_addr[1:0]`** identifying the byte position within the word.

A write occurs on the rising edge of **`i_clk`** when **`i_wren`** is high. Each bit of **`i_bmsk`** independently enables the corresponding byte lane. Disabled lanes keep their previous values.

Table 9: Examples of byte enable masks.

`i_bmsk`	Bytes written	Example
`4'b0000`	0	No byte is modified.
`4'b0001`	1	Write byte lane 0.
`4'b0010`	1	Write byte lane 1.
`4'b0100`	1	Write byte lane 2.
`4'b1000`	1	Write byte lane 3.
`4'b0011`	2	Write the lower two byte lanes.
`4'b1100`	2	Write the upper two byte lanes.
`4'b1111`	4	Write the complete word.

Other byte-mask combinations are also valid. Each asserted bit controls its corresponding byte lane.

The memory uses the parameters **`START_ADDR`** and **`ADDR_WIDTH`**. The **`START_ADDR`** shall be word-aligned and **`ADDR_WIDTH`** shall be at least 2. The valid memory region is:

$$\mathrm{START_ADDR} \leq \mathrm{i_addr} < \mathrm{START_ADDR} + 2^{\mathrm{ADDR_WIDTH}}.$$

The out-of-range writes are ignored and out-of-range reads return zero. For a valid address:

$$\mathrm{offset} = \mathrm{i_addr} - \mathrm{START_ADDR}.$$

The read port is asynchronous and always active. **`o_rdata`** follows the addressed word through combinational logic and does not require a clock edge or read enable.

If a read and write access the same word, **`o_rdata`** shows the previous value before the rising edge and the updated value after the write completes.

Table 10: Parameters of the asynchronous read memory model.

Parameter	Type	Description
START_ADDR	32-bit value	Base address of the memory region.
ADDR_WIDTH	integer	Number of byte-address bits. The memory region contains $2^{\text{ADDR_WIDTH}}$ bytes.

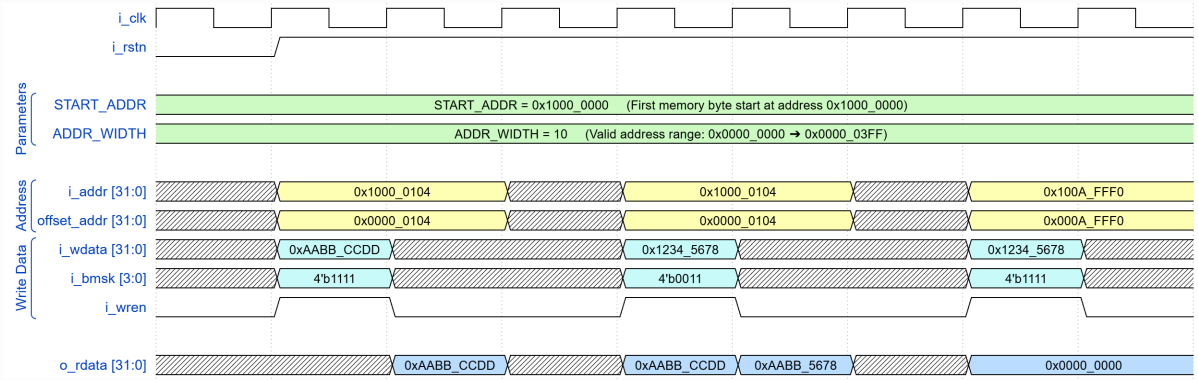


Figure 7: Example asynchronous memory timing behavior.

The memory uses little-endian byte ordering. Within a 32-bit word, the least significant byte is stored at the lowest address and the most significant byte at the highest address.

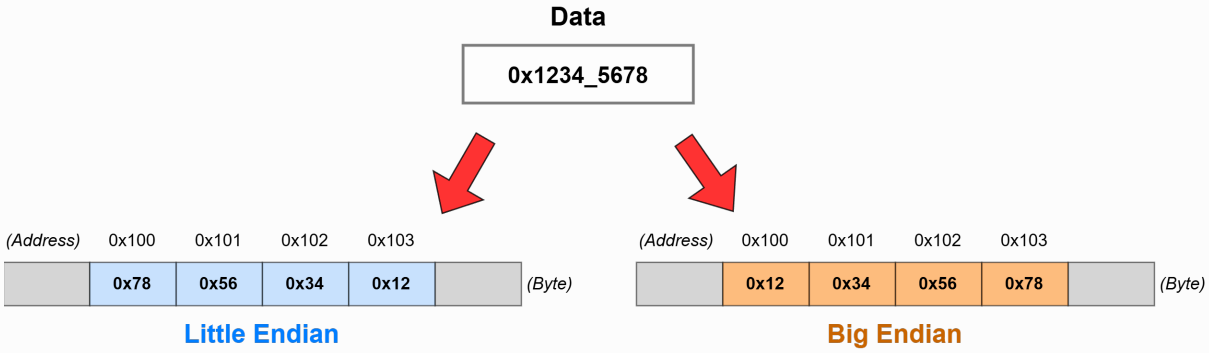


Figure 8: Little-endian byte ordering of a 32-bit word.

PROBLEM

Students implement the asynchronous read memory according to the specified interface, addressing, byte-enable, and little-endian behavior. Students are also encouraged to write their own testbench to verify the design.

4.7. Exercise 7: Data Memory with Synchronous Read

This exercise uses the same memory organization, write behavior, byte-enable rules, address range, and parameters as the asynchronous memory in the previous exercise. The difference is that the read port is synchronous and controlled by a read enable signal.

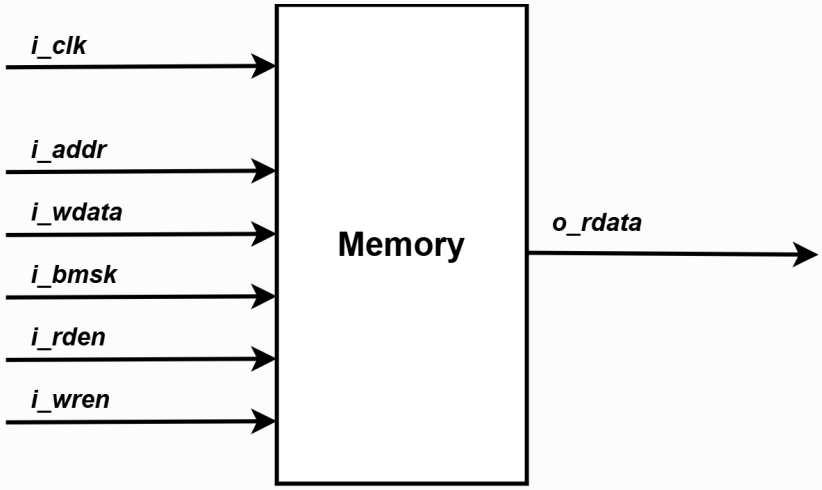


Figure 9: Synchronous read memory input/output signal specification

Table 11: Input and output signals of the synchronous read memory.

Signal name	Width	Direction	Description
`i_clk`	1	input	Clock of the memory.
`i_addr`	32	input	Byte address of the memory access.
`i_wdata`	32	input	Data to be written.
`i_bmsk`	4	input	Byte enable mask for the four byte lanes.
`i_rden`	1	input	Read enable.
`i_wren`	1	input	Write enable.
`o_rdata`	32	output	Registered read data output.

The write port behaves as in the previous exercise. When **`i_wren`** is high, the selected byte lanes are updated on the rising edge of **`i_clk`**.

The read port is synchronous. When **`i_rden`** is high, the word selected by **`i_addr`** is sampled on the rising edge of **`i_clk`** and transferred to **`o_rdata`**. The read therefore has one clock cycle of latency. When **`i_rden`** is low, **`o_rdata`** keeps its previous value.

The address range and out-of-range behavior are the same as in the previous exercise. An out-of-range read returns zero, while an out-of-range write is ignored.

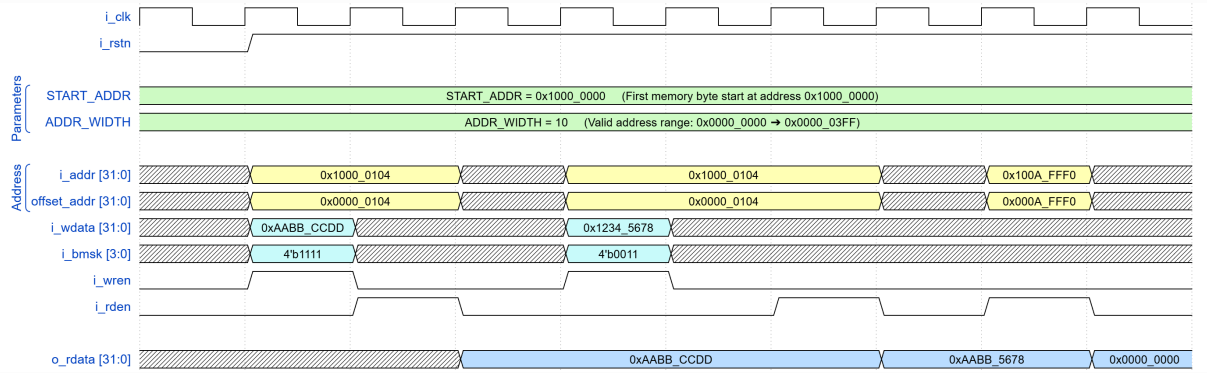


Figure 10: Example synchronous memory timing behavior.

---

*The behavior when **`i_rden`** and **`i_wren`** are both high for the same memory access is intentionally left undefined in this exercise. Students shall choose and implement a consistent default behavior for this case, such as read-before-write or write-before-read, and document their choice.*

PROBLEM

Students implement a synchronous-read memory using the same parameters and memory organization as the previous exercise. The design should support registered reads, byte-enabled writes, address validation, and defined simultaneous read/write behavior.

This is an optional extra-credit exercise intended to prepare students for later milestones. Completion is not required.

5. Assessment

- Grade: 10% of the Lab overall score
- Time: 2 weeks
- Requirement: Students submit report explaining designs and simluation & verification results
- Submission: Student submite report on Server